

AGENT DESIGN DOCUMENT

Personal Productivity & Task Execution Agent

1. Problem Statement

Knowledge workers manage their day across scattered surfaces — meeting notes, chat threads, sticky notes, and a task list that is rarely kept current. General-purpose chatbots can discuss productivity in the abstract, but they cannot see or change a user's actual tasks and notes, so any plan they produce is disconnected from reality and quickly goes stale.

This project builds a tool-using AI agent that closes that gap. Instead of only talking about tasks, the agent reads and writes a real task and note store through a defined set of tools, so its answers (“what should I work on today”, “what's overdue”, “turn these meeting notes into tasks”) are grounded in the user's actual data rather than invented. The agent is explicitly scoped to organize, extract, plan, and report on a user's work — it is not a general chatbot, and it never takes an irreversible action without explicit human approval.

2. Users

- **Primary user** — Individual knowledge workers and students who keep an informal task list and want it converted into something structured, prioritized, and plannable — the primary persona for this build.
- **Secondary user** — Freelancers juggling multiple concurrent client engagements, who need a fast way to turn meeting notes and client messages into tracked tasks and follow-ups.
- **Tertiary / future user** — A future team lead or manager persona, who would use the weekly report and reminder tools to track a small team's progress (out of scope for this build's data model, which is single-user, but the tool design does not preclude it).

All users in this build are assumed to be a single authenticated individual per running instance; there is no multi-tenant user model yet (see Security Considerations).

3. Use Cases

The agent is designed against the following representative requests, each of which exercises a different part of the decision logic:

- “Create three tasks from these meeting notes.” — structured extraction, then gated batch task creation.
- “Show me all high-priority tasks due this week.” — filtered read-only retrieval, no approval needed.
- “Prepare a daily work plan using my pending tasks.” — multi-factor scheduling over stored data.
- “Summarize these notes and identify decisions and action items.” — structured LLM extraction with a fixed output schema.

- “Create a reminder for the project review.” — single gated write action.
- “Search my saved notes for information about the marketing campaign.” — keyword/relevance search.
- “Draft a follow-up email based on these meeting notes.” — gated, LLM-drafted artifact; never actually sent.
- “Mark the website task as complete.” — gated single-record update with ID resolution from context.
- “Prepare a weekly productivity report.” — aggregation and next-period recommendation.
- “Find tasks that are overdue and recommend what I should work on first.” — combined detection + prioritization.
- “Explain the difference between high and critical priority.” — a request the agent must answer directly, without invoking any tool.

4. Agent Responsibilities

Within the scope defined above, the agent is responsible for:

- Interpreting each user request and deciding whether it requires a tool at all, versus a direct factual answer.
- Selecting the single most specific tool for a request, or sequencing multiple tools for genuinely multi-step workflows (meeting notes → tasks; daily planning; weekly review).
- Validating every tool argument against a strict schema before execution, and rejecting or asking for clarification on malformed input rather than guessing.
- Pausing and requesting explicit human approval before any write, update, or externally-visible action.
- Reviewing tool output for basic sufficiency before using it to compose a response, and never fabricating task IDs, counts, dates, or content a tool did not actually return.
- Maintaining short-term session memory so that references like “the second one” resolve correctly to a specific record within the same conversation.
- Logging every step of every run — request, tools called, arguments, results, approval status, timing, and outcome — for later review.
- Reporting errors in plain language and stopping cleanly rather than retrying indefinitely or silently failing.

5. Agent Boundaries

The agent is deliberately restricted from doing the following, regardless of how a request is phrased:

- It may not execute any gated action — `update_task`, `complete_task`, `create_reminder`, `draft_followup_email`, or multi-task creation — without an explicit Approve from the user. This is enforced in code (a fixed allow-list), not only through prompt instructions, so it cannot be bypassed by the model ignoring guidance.
- It may not actually send email, or perform any other externally-visible action outside the local data store; “send” requests are always simulated and clearly labeled as drafts.
- It may not invent data: task IDs, note contents, dates, or counts must always trace back to an actual tool result.
- It may not call a tool with the exact same name and arguments more than once within a single run (duplicate-call detection), and may not exceed a fixed maximum number of reasoning steps per run — both are hard stops, not suggestions.

- It may not access data belonging to another user or session; session memory is scoped to a single chat session and never shared.
- It may not expose internal chain-of-thought, stack traces, or API keys in any response; only short operational status and final outcomes are shown.
- It does not have general chatbot behavior — requests unrelated to task/note/plan management are outside its intended scope even if the underlying model could technically answer them.

6. Tool Catalogue

The agent has thirteen tools: eight required by the core specification and five additional tools for reminders, overdue detection, note summarization, email drafting, and weekly reporting. Every tool validates its input against a Pydantic schema and returns a uniform result envelope (success flag, data payload, error message, approval flag) so the controller can treat all tools identically.

Tool	Purpose	Key Inputs	Approval
create_task	Create a single new task.	title, description, priority, due_date, tags	No
list_tasks	List tasks filtered by status, priority, due date, or tag.	status, priority, due_before, tag	No
update_task	Modify fields on an existing task.	task_id, fields to change	Yes
complete_task	Mark a task as Completed.	task_id	Yes
search_notes	Keyword search over saved notes with relevance scoring.	query, category, date range	No
save_note	Save a new note.	title, content, category, tags	No
extract_meeting_actions	Extract a structured summary, decisions, action items, owners, deadlines, and open questions from meeting notes (LLM-backed).	transcript	No*
generate_work_plan	Build an ordered daily schedule from pending tasks, weighted by priority, deadline, and status.	available_hours, priorities, plan_date,	No
detect_overdue_tasks	Find tasks past their due date that are not completed or cancelled.	as_of (optional)	No
create_reminder	Create a lightweight reminder task.	title, due_date, note	Yes
summarize_notes	Retrieve recent notes (optionally by category) for summarization.	category, max_notes	No
draft_followup_email	Draft a follow-up email from meeting notes (LLM-backed; never sent).	meeting_notes, recipient_name, tone	Yes
generate_weekly_report	Summarize completed / overdue / blocked tasks and recommend next-week priorities.	week_ending (optional)	No

** extract_meeting_actions is read-only (it does not write to the task/note store), so it is not gated on its own; the task-creation step that typically follows it is gated as a batch action.*

7. State Model

Per-request state (AgentRunState)

Created fresh for every user message and discarded once the run finishes: the original request text, a running execution log, the current status (idle / thinking / selecting_tool / waiting_approval / executing_tool / responding / error), the number of steps taken so far, the most recent tool result, any pending approval object, and a history of tool-call signatures used for duplicate/loop detection.

Session state (SessionMemory)

Held for the lifetime of the chat session (in the UI's session store): the recent message history used to give the model conversational context; the most recently listed tasks, so that references such as “the second one” can be resolved to a specific task ID; the most recent output of each tool, for follow-up questions; and any preferences the user has stated during the session (for example, typical working hours).

Persistent state (database)

Everything that must survive beyond a single session is written to the database through a single repository layer: tasks, notes, and the full structured log of every run. The agent and tool layers never access the database directly — all reads and writes go through this repository, which is the only layer that can raise a storage-level error.

8. Approval Model

Every action that changes stored data or produces an externally-visible artifact requires an explicit human decision before it runs. The gate is enforced twice: once in the system prompt (so the model is discouraged from claiming an action already happened) and once in code, against a fixed list of gated tool names — so approval cannot be skipped even if the underlying model ignores its instructions.

Action	Why it is gated	Approval interface shows
update_task	Silently changing task data could hide edits the user did not intend.	Tool name, arguments, expected effect, Approve / Reject
complete_task	Marking work done is easy to reverse in name only — the user must confirm the task is actually finished.	Tool name, arguments, expected effect, Approve / Reject
create_reminder	A reminder is a standing commitment; the user should confirm timing before it is created.	Tool name, arguments, expected effect, Approve / Reject
draft_followup_email	Email drafts represent an outward-facing artifact; even though sending is simulated only, the content must be reviewed first.	Tool name, arguments, expected effect, Approve / Reject
Creating more than one task in a run	Bulk creation (e.g. from meeting notes) has a larger blast radius than a single task and should not happen silently.	Proposed task list, Approve / Reject

When an action is approved, the tool executes and the result feeds back into the conversation as normal. When an action is rejected, the tool never runs, the rejection is recorded in the execution log, and control returns to the user with a plain confirmation that no changes were made.

9. Error Strategy

The agent is built to fail safely and visibly rather than silently or destructively. Specific handling includes:

- Missing or invalid API key: detected before any request is attempted; the user sees a direct, actionable message instead of a raw provider error.

- Invalid model response (malformed JSON, unparseable tool arguments): caught and converted into a plain-language error instead of crashing the run or being passed through to the data layer.
- Invalid tool arguments: every tool validates input against its Pydantic schema before running; validation failures are returned as a clear message naming the offending field, not a stack trace.
- Unknown task ID: checked explicitly before any update/complete operation and reported as “Unknown task ID” rather than a database exception.
- Database failures: isolated to the repository layer, wrapped in a dedicated exception type, and surfaced to the UI as a readable message.
- Empty user input: rejected before any LLM call is made.
- Tool execution timeout: every tool call runs under a bounded timeout (default 30s); a timeout is treated as a failed attempt and can be retried up to a fixed limit (default 2 retries) before being reported as an error.
- Repeated failures / loops: a duplicate-call guard stops the run if the same tool would be called with the same arguments twice in one run, and a hard step ceiling (default 8) stops any run that has not converged.
- Unsupported requests: if no tool matches and the request cannot be answered directly, the agent says so rather than guessing or calling an unrelated tool.

In all cases, the error shown to the user is a short, plain-language description of what went wrong and, where relevant, a suggested next step. Raw exceptions, stack traces, and API keys are never rendered in the interface; the full technical detail is written only to the execution log for later review.

10. Security Considerations

Secrets

API keys are read exclusively from environment variables at process start and are never hard-coded, logged, or included in any tool result, execution log entry, or model prompt. The execution log explicitly excludes API keys and raw chain-of-thought by design, recording only operational events (which tool ran, with what arguments, for how long, and with what outcome).

Permissions

The tool layer is the only path by which the agent can affect stored data, and every write-capable tool is bound by the approval model described above — the agent cannot escalate its own permissions or bypass a gate through prompt manipulation, because the gate check happens in code against a fixed list, not inside the model's own reasoning.

User data

All task and note data stays in the configured local database; no task or note content is sent anywhere except to the configured LLM provider as part of answering the user's own request. Session memory (recent messages, referenced tasks, preferences) is held only for the duration of the session and is not shared across sessions or users. Draft artifacts such as follow-up emails are generated but never actually transmitted, so no external send-side data exposure occurs from that tool.

Input handling

Every tool argument is validated and type-checked before it can reach the database layer, which constrains the range of malformed or malicious input that can reach persistence. Because the ORM layer uses parameterized queries throughout, there is no free-text SQL construction from user input.